Understanding Deep Learning: Chapter 1 - Introduction

Simon J. D. Prince
University of Bath
MIT Press 2023

Abstract

This chapter introduces deep learning as the study of models with many layers of computation that are learned from data. It defines the supervised learning problem, explains why depth is useful, and reviews the historical developments that made deep networks practical. The chapter closes with an overview of the book and guidance on how to read it. No prior background in machine learning is assumed beyond basic linear algebra, calculus, and probability.

Keywords: deep learning, introduction, supervised learning, neural networks, textbook

1 Introduction

Deep learning is a subfield of machine learning that studies models composed of many layers of simple computations. Each layer transforms its input into a representation that is more useful for the task at hand, and the parameters of all layers are learned jointly from data. The word deep refers to the number of layers, not to any claim about the depth of understanding achieved by the model.

This book is about how such models work. It explains the building blocks, the training algorithms, and the design choices that determine whether a network learns well or fails. The emphasis is on ideas that generalize across applications, not on the details of any particular benchmark. Figure 1 shows the place of deep learning within artificial intelligence and machine learning.

A reader who finishes the book should be able to implement a small deep network from scratch, understand why it trains or does not train, and read the research literature with confidence. The book assumes familiarity with vectors, matrices, derivatives, and basic probability, but it does not assume prior exposure to neural networks.

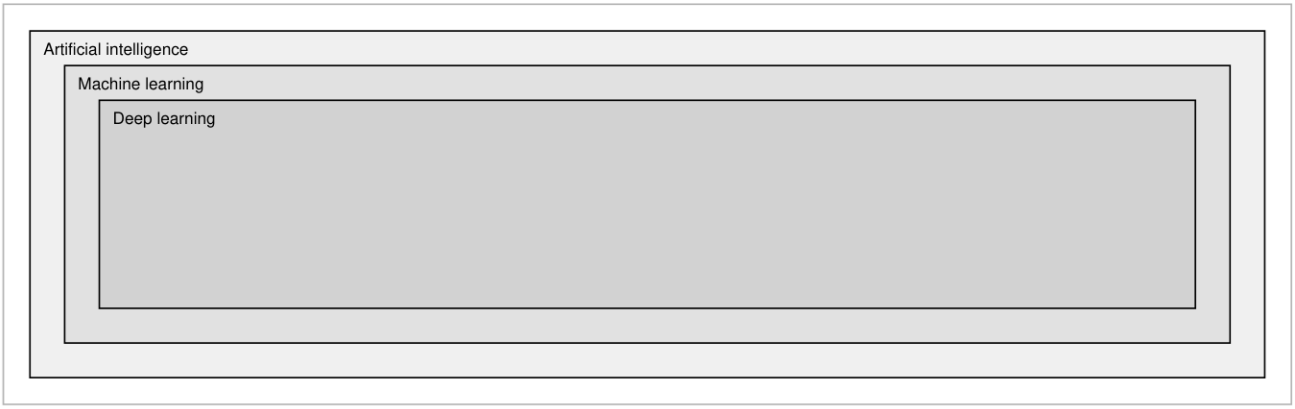


Figure 1: Deep learning as a subfield of machine learning, which is itself a subfield of artificial intelligence.

1.1 What is Deep Learning?

In supervised learning we are given a training set of input-output pairs (x_i, y_i), and we want to find a function f that maps inputs to outputs and generalizes to new inputs. A deep network represents f as a composition of layers, f(x) = f_L(f_(L-1)(...f_1(x)...)), where each layer has its own parameters. The composition is what gives the model its expressive power: a deep network can represent functions that would require an exponentially large shallow network to approximate.

The parameters are chosen by minimizing a loss function that measures the mismatch between predictions and targets, as in Equation (1). For regression the loss is usually squared error; for classification it is usually cross-entropy. The optimizer adjusts the parameters by gradient descent, and the gradients are computed efficiently by backpropagation.

Deep learning differs from classical machine learning in that the representation itself is learned. In a classical pipeline, a human engineer designs features and the model learns a simple mapping from features to outputs. In a deep network, the features are learned by the earlier layers and the final layer learns the mapping. This end-to-end approach is the main reason deep networks outperform classical pipelines when data is plentiful.

A useful way to think about a deep network is as a stack of representation learners. Each layer maps its input into a new coordinate system in which the prediction task is simpler. The lower layers are shared across tasks, while the upper layers specialize. This division of labor is why a network trained on one task can often be fine-tuned on another with few labeled examples, and it is the idea behind the transfer learning methods discussed in later chapters.

Deep learning is not limited to supervised learning. Unsupervised methods learn structure from unlabeled data, and reinforcement learning learns from rewards. The supervised setting is emphasized here because it is the cleanest way to introduce the mechanics of layers, losses, and gradients, and because most of the techniques carry over to the other settings. Figure 3 shows the computation performed by a single neuron.

$$L(\theta) = \frac{1}{N} \sum_{i} \ell\left(f(x_i; \theta), y_i\right) \tag{1}$$

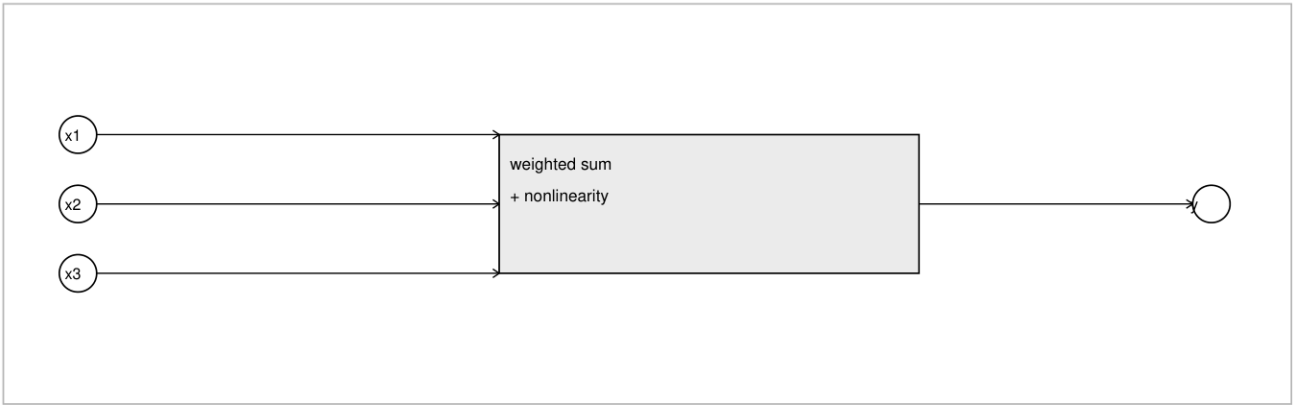


Figure 3: A single neuron computes a weighted sum of its inputs, adds a bias, and applies a nonlinearity.

1.2 A Brief History

The idea of learning from data with layered models is old. The perceptron, introduced in the late 1950s, was a single-layer linear classifier, and its limitations were famously analyzed in the 1960s. Multilayer networks and backpropagation became popular in the 1980s, but they were difficult to train at scale and fell out of favor in the 1990s.

Three developments revived the field. First, large labeled datasets such as ImageNet made it possible to train models with millions of parameters. Second, graphics processing units made the arithmetic fast enough to train such models in reasonable time. Third, improved activation functions, initialization schemes, and regularization methods made deep networks trainable.

The turning point was the AlexNet result on ImageNet in 2012, which reduced the top-5 error rate to 15.3 percent and beat the previous best by a large margin. The network had 60 million parameters and eight layers, and it was trained on two GTX 580 GPUs for five to six days. Subsequent work introduced residual connections, which made it possible to train networks with more than one hundred layers, and attention-based architectures, which now dominate language modeling.

The years after 2012 brought a steady sequence of architectural innovations. Batch normalization made deeper convolutional networks easier to optimize, residual connections removed the degradation problem, and generative adversarial networks introduced a new training paradigm. Recurrent networks with attention dominated sequence modeling until 2017, when the Transformer replaced recurrence with self-attention. Transformer-based models then scaled from millions to hundreds of billions of parameters, and the same recipe was adapted to images, audio, and multimodal data. This history matters because many current design choices are responses to practical failures of earlier architectures rather than purely theoretical advances.

1.3 Why Depth Matters

Depth is useful because many functions of interest have a compositional structure. A face is composed of eyes, a nose, and a mouth; a sentence is composed of words, phrases, and clauses. A deep network can reuse intermediate representations across tasks, so the number of parameters needed to represent a compositional function grows more slowly with depth than with width.

Depth also enables abstraction. Earlier layers learn low-level features such as edges and textures, middle layers learn parts and motifs, and later layers learn objects and concepts. This hierarchy is not imposed by hand; it emerges from training on data. Visualization studies of convolutional networks provided some of the earliest evidence for this behavior.

Depth is not free. Deeper networks are harder to optimize, more expensive to train, and more prone to overfitting when data is limited. The rest of the book is largely about the techniques that make deep networks trainable: careful initialization, normalization, residual connections, and regularization.

A second reason depth helps is parameter efficiency. A deep network can represent a function as a composition of simpler functions, and for many structured functions the number of parameters needed grows linearly with depth, whereas a shallow network may need exponentially many units in its width. This argument is informal, but it matches practice: deep convolutional networks outperform shallow ones with comparable parameter counts on image recognition, and deep transformers outperform shallow ones on language tasks at fixed parameter budgets. Depth is therefore not merely a way to add capacity; it changes how capacity is organized.

1.4 Overview of the Book

The book is organized into four parts. Part I introduces the supervised learning problem and the basic building blocks of neural networks. Part II covers training: loss functions, gradients, backpropagation, and optimization. Part III covers architectures for images, sequences, and sets, including convolutional networks, recurrent networks, and transformers. Part IV covers advanced topics such as generative models, reinforcement learning, and ethics.

Each chapter includes worked examples, exercises, and pointers to the literature. The exercises are graded from conceptual questions to implementation tasks, and solutions are available to instructors. The book's website hosts code notebooks that reproduce the figures.

The chapters are designed to be read in order, but they are also modular. A reader interested in training can start with Part II and return to Part I for notation; a reader interested in applications can read Part III after Chapter 2. Each part ends with a short summary and a set of exercises that connect the ideas to practice. Supplementary material on the website includes datasets, code, and a glossary.

2 The Supervised Learning Problem

This section formalizes the supervised learning problem and introduces the notation used throughout the book. We distinguish between the training loss, which is minimized on the training set, and the test loss, which measures generalization to unseen data.

The central challenge is that minimizing training loss does not guarantee low test loss. A model with enough parameters can memorize the training set, achieving zero training loss while generalizing poorly. Regularization, data augmentation, and early stopping are all ways to control this trade-off.

2.1 Loss Functions

A loss function assigns a scalar cost to a prediction. For regression, the squared error loss is differentiable and easy to optimize, but it is sensitive to outliers. For classification, the cross-entropy loss is the standard choice because it is the negative log-likelihood of the correct class under a softmax model.

The choice of loss should reflect the cost of errors in the application. In medical diagnosis, a false negative may be much worse than a false positive, and the loss should encode that asymmetry. In many cases the right loss is not the one that is easiest to optimize.

Loss functions also determine the scale of the gradients. Squared error produces gradients proportional to the prediction error, so learning slows as the error shrinks; cross-entropy produces gradients proportional to the difference between the predicted probability and the target, which stays informative even when the model is confident and wrong. This difference is one reason cross-entropy is preferred for classification, and it illustrates a general principle: the loss should be chosen together with the output activation, not independently. Table 2 lists the

notation used throughout the book. The parameters themselves are updated by gradient descent, as in Equation (2).

A loss function is usually averaged over the training set, so its scale depends on the number of examples. When comparing losses across datasets, it is therefore better to report per-example averages than sums. The same care applies to reported accuracies: a number without its dataset, split, and evaluation protocol is difficult to interpret.

theta <- theta - eta * grad_theta L(theta) (2)

Table 2: Notation used throughout the book.

Symbol	Meaning
x	Input example
y	Target or label
f	Model function
theta	Model parameters
L	Loss function
eta	Learning rate
grad	Gradient operator
D	Training dataset

2.2 Training and Testing

We split the available data into a training set, a validation set, and a test set. The training set is used to fit the parameters, the validation set is used to choose hyperparameters, and the test set is used only once to estimate generalization. Using the test set for model selection is a common mistake that leads to optimistic estimates.

When data is scarce, cross-validation provides a more efficient use of the available examples. The training data is divided into folds, and each fold is held out in turn while the model is trained on the remaining folds. The average validation loss across folds is then used for model selection.

Choosing the split between training, validation, and test data is itself a modeling decision. If the same distribution is used for all three, performance estimates can be optimistic, because subtle leakage through preprocessing or duplicate examples inflates test accuracy. A clean protocol fixes the split before any model is trained, tunes hyperparameters only on the validation set, and reports the test set once. When data is scarce, cross-validation reuses examples across folds at the cost of additional computation.

A final practical issue is distribution shift. A model trained on one population may encounter a different population after deployment, and accuracy can degrade sharply even when the new data looks similar to a human. Monitoring input statistics and re-evaluating on recent data are therefore part of the lifecycle of a deployed model, not a one-time check performed before release.

3 Summary

Deep learning studies models with many layers of learned representation. Depth provides compositionality and abstraction, but it also makes optimization harder. The field's recent success depends on large datasets, fast hardware, and training techniques that keep deep networks stable.

The next chapter introduces the simplest deep network, the fully connected network, and shows how to train it on a small classification problem. Later chapters build on this foundation to cover convolutional networks, sequence models, and transformers.

The chapter has introduced the vocabulary and the questions that the rest of the book answers. The key idea is that depth turns a hard function into a composition of easier functions, and that learning succeeds when data, optimization, and regularization are balanced. The remaining chapters make each of these ingredients precise.

Table 1: Milestones in the development of deep learning discussed in this chapter.

Year	Development	Impact
1958	Perceptron	Single-layer learning
1986	Backpropagation	Multilayer training
2012	AlexNet	ImageNet breakthrough
2015	ResNet	Networks beyond 100 layers
2017	Transformer	Attention-based sequence models

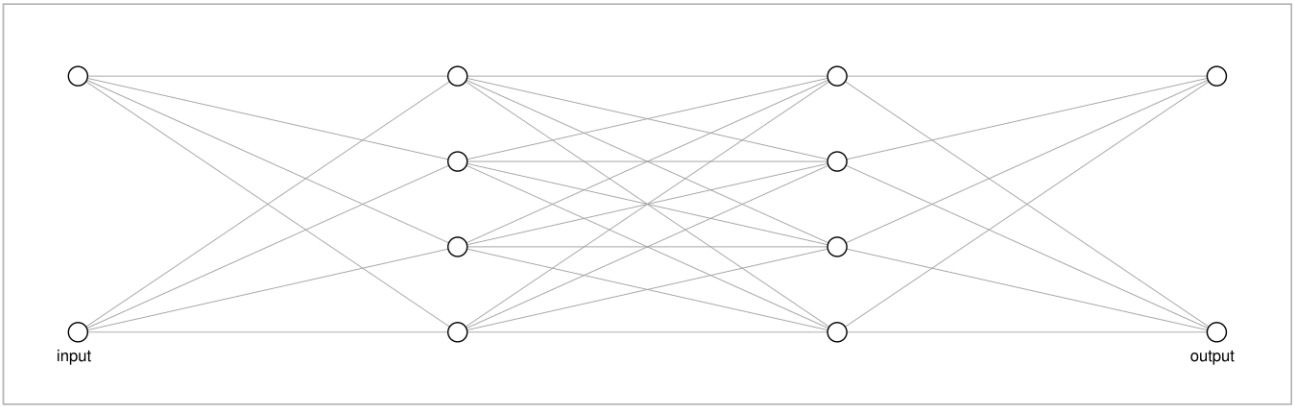


Figure 2: A feed-forward network with an input layer, two hidden layers, and an output layer.

References

[1] Krizhevsky, Sutskever, and Hinton. ImageNet Classification with Deep Convolutional Neural Networks. NeurIPS, 2012.

[2] He et al. Deep Residual Learning for Image Recognition. CVPR, 2016.

[3] Vaswani et al. Attention Is All You Need. NeurIPS, 2017.

[4] Rumelhart, Hinton, and Williams. Learning Representations by Back-Propagating Errors. Nature, 1986.

[5] Rosenblatt. The Perceptron: A Probabilistic Model for Information Storage and Organization in the Brain. Psychological Review, 1958.

[6] LeCun, Bengio, and Hinton. Deep Learning. Nature, 2015.

[7] Goodfellow, Bengio, and Courville. Deep Learning. MIT Press, 2016.

[8] Devlin et al. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. NAACL, 2019.

[9] Deng et al. ImageNet: A Large-Scale Hierarchical Image Database. CVPR, 2009.

[10] Glorot and Bengio. Understanding the Difficulty of Training Deep Feedforward Neural Networks. AISTATS, 2010.

A Appendix: Further Reading

Readers who want a broader historical perspective should consult the review by LeCun, Bengio, and Hinton (2015) and the textbook by Goodfellow, Bengio, and Courville (2016). Both cover the mathematical background in more detail than this chapter, and the latter includes a thorough treatment of regularization and optimization.

For a hands-on introduction, the notebooks that accompany this book implement every model in the text using a small autodifferentiation library. The notebooks are designed to run on a laptop CPU and are intended to be modified: changing the number of layers, the activation function, or the learning rate is the fastest way to build intuition for the ideas in the following chapters.